

Yes! This is one of the **most important security concepts** in authentication. 🗝️

Why don't we store the actual password?

Suppose you register with:

```
email: gopi@gmail.com  
password: hello123
```



A bad database would look like:

```
users  
-----  
email          password  
gopi@gmail.com hello123
```



At first this seems convenient because when you log in, the server can simply do:

```
if (enteredPassword === user.password) {  
  // login  
}
```

But **this is extremely dangerous.**

Imagine the database gets leaked 😬

If an attacker gets the database:

```
gopi@gmail.com → hello123
```



They immediately know your password.

And the bigger problem is **password reuse**.

If you use:

```
hello123
```



for:

- Gmail
- Instagram
- GitHub
- Banking
- Shopping sites

then one leaked database can potentially expose all those accounts.

So we store a password hash

Instead of:

hello123



the database stores something like:

\$argon2id\$v=19\$m=65536,t=3,p=4\$...



Conceptually:

```
password
  ↓
password hashing algorithm
  ↓
password hash
```



The important property is:

You cannot practically reverse the hash to recover the original password.

So even if the database leaks:

gopi@gmail.com
\$argon2id\$...



the attacker doesn't simply get:

hello123



But then how does login work?

This is the really cool part.

You might wonder:

"If we don't know the original password anymore, how can we check whether the user entered the correct password?"

We hash/verify the entered password again.

Suppose registration:

User chooses:

hello123



Server:

hello123

↓

Argon2

↓

HASH_A



Database:

passwordHash = HASH_A



Later, during login:

User enters:

hello123



Server:

hello123

↓

Argon2 verification

↓

compare against stored hash



If it matches:

✅ Correct password



If user enters:

hello456



then:

hello456

↓

verification

↓

✗ Doesn't match stored hash



One important correction: hashing ≠ encryption

This is a common interview question.

Encryption

plaintext

↓

encrypt

↓

ciphertext

↓

decrypt

↓

plaintext



Encryption is designed to be **reversible** when you have the key.

Example:

"hello123"

↓ encryption

"X7&kP..."

↓ decryption + key

"hello123"



Password hashing

password

↓

hash

↓

hash



There is intentionally **no normal "decrypt" operation**.

So:

Passwords should be **hashed**, not encrypted for storage.

But wait... aren't hashes crackable?

This is where **salt + slow password hashing algorithms** come in.

A production password hash typically includes a **salt**.

Conceptually:

```
password + random salt
      ↓
    Argon2/bcrypt
      ↓
      hash
```



So two users who happen to choose:

```
hello123
```



don't necessarily end up with the same stored hash.

For example:

```
User A:
hello123 + saltA
      ↓
    hashA
```



```
User B:
hello123 + saltB
      ↓
    hashB
```

The salts make precomputed attacks such as rainbow tables much less useful.

And algorithms such as **Argon2id** are deliberately designed to make large-scale password guessing expensive.

Why not just use SHA-256?

Another classic interview trap. 😊

You might think:

```
sha256(password)
```

Problem: SHA-256 is designed to be **very fast**.

That's great for many cryptographic applications.

But for passwords, we actually want password hashing to be **deliberately expensive**.

An attacker might have:

```
10,000,000 possible passwords
```



If the hashing operation is extremely fast, they can try huge numbers of guesses.

Password-specific algorithms such as:

- Argon2id
- bcrypt
- scrypt
- PBKDF2

are designed to make guessing more expensive.

The complete picture

Registration

```
Browser
|
| password = "hello123"
▼
Server
|
| hash password
▼
Argon2id
|
▼
passwordHash
|
```



Database

Database:

email: gopi@gmail.com

passwordHash:
\$argon2id\$...

Not:

password: hello123 ❌

Login

Browser

password = "hello123"

Server

verify entered password
against stored hash

Database

passwordHash

Server

valid → create session ✅

invalid → reject ❌

And importantly:

The server doesn't need to know the user's original password after registration.

It only needs to be able to **verify whether a supplied password matches the stored hash.**



Interview-ready answer

If they ask:

"Why should passwords be hashed instead of stored directly?"

You can say:

"Passwords should never be stored in plaintext because a database breach would immediately expose users' passwords, including passwords they may have reused elsewhere. Instead, we store a salted password hash using a password-specific hashing algorithm such as Argon2id or bcrypt. During login, we verify the supplied password against the stored hash rather than decrypting it. Hashing is intentionally one-way, and password-hashing algorithms are designed to make brute-force guessing expensive."

That's the answer I'd give in an interview. 🔥

And one tiny distinction to keep in your head:

Password → hash → DB

Login password → verify against hash → session

Not:

Password → encrypt → DB → decrypt.



whats SHA-256 or salt A or i didnt understnad exactly

Yep 😊 — let's **slow this way down**, because three different concepts got mixed together:

1. **SHA-256**
2. **Hashing**
3. **Salt**

Once these click, password storage becomes very easy.

1. First: what is SHA-256?

SHA-256 is a hashing algorithm.

Think of a hash function as a machine:

HASH MACHINE

"hello123" → "ef92b778..."



You give it some input, and it produces a fixed-looking string called a **hash**.

For example:

"hello123"
↓
SHA-256
↓
ef92b778baf771e...



The important property is:

"hello123" → same SHA-256 hash every time



So:

SHA256("hello123")
=
ef92b778...



every time.

2. Why is SHA-256 called a hash?

Because it transforms:

INPUT
↓
HASH



and you normally **don't reverse it**.

For example:

"hello123"
↓
SHA-256
↓
"ef92b778..."



You don't have:

```
"ef92b778..."
```

↓

```
SHA-256
```

↓

```
"hello123" ❌
```



That's not how hashing works.

3. Then what's the problem with SHA-256 for passwords?

Here's where it gets interesting.

Imagine:

```
User password:
```

```
hello123
```



We do:

```
hello123
```

↓

```
SHA-256
```

↓

```
HASH
```



And save:

```
database:
```

```
passwordHash = HASH
```



Looks safe, right?

Better than plaintext, but not sufficient for password storage.

Why?

Because SHA-256 is **very fast**.

An attacker who steals your database can try:

```
password1
```

```
password123
```

```
hello123
```

```
qwerty
```



```
letmein  
...
```

and calculate SHA-256 for millions/billions of guesses very quickly.

If one matches the stolen hash:

```
SHA256(guess) === stolenHash
```



🎯 They found the password.

So for passwords, we want something that makes each guess **expensive**.

That's why we use algorithms designed specifically for passwords, such as:

- Argon2id
- bcrypt
- scrypt
- PBKDF2

4. Now: what's a SALT? 🧂

This is probably the confusing part.

A **salt** is just a random value generated for a particular password before hashing it.

Suppose:

```
Password = hello123
```



The server generates a random salt:

```
Salt A = X7k92p
```



Then conceptually:

```
hello123 + X7k92p  
  ↓  
  Argon2id  
  ↓  
  Hash A
```



The database stores the information needed for verification, including the salt and resulting hash.

Conceptually:

```
user
├─ passwordHash = Hash A
└─ salt = X7k92p
```



The salt is **NOT** the password.

And it's **not a secret** like a password or encryption key.

5. Why do we need the salt?

Here's the key example.

Imagine two people:

Gopi's password = hello123



Priya's password = hello123

Without salt:

```
hello123
  ↓ SHA-256
HASH123
```



Both users get:

```
Gopi → HASH123
Priya → HASH123
```



An attacker sees:

```
Gopi → HASH123
Priya → HASH123
```



They immediately know both users have the same password.

More importantly, attackers can use **precomputed tables** of common passwords and their hashes.

6. With salt

Now:

Gopi

Password:
hello123



Random salt:
A8x91

hello123 + A8x91

↓

Argon2id

↓

HASH_A

Priya

Password:
hello123



Random salt:
K72pq

hello123 + K72pq

↓

Argon2id

↓

HASH_B

So:

Gopi → HASH_A
Priya → HASH_B



Even though:

Gopi's password = hello123
Priya's password = hello123



their stored hashes are different.

7. So what's "Salt A" and "Salt B"?

When I previously said:

```
User A:  
hello123 + saltA
```



and:

```
User B:  
hello123 + saltB
```



I was **just giving the random salts names**.

It doesn't mean there is literally something called:

```
saltA  
saltB
```



It's just:

```
User A's random salt  
User B's random salt
```



For example, the actual values could look more like:

```
User A:  
salt = 8f3a91c2...  
  
User B:  
salt = d71b04e9...
```



8. Very important: the salt doesn't have to be secret

This is another thing that confuses people.

You might think:

"If the salt is stored in the database, can't the attacker see it?"

Yes.

That's okay.

The security comes from:

```
random salt
+
slow password-hashing algorithm
+
unknown password
```



The salt's purpose is **not to hide the password**.

Its purpose is mainly to ensure that identical passwords don't produce identical hashes and to defeat precomputed/rainbow-table attacks.

9. Now let's see the real login process

Suppose during signup:

```
Password = hello123
```



Server generates:

```
salt = A8x91
```



Then:

```
hello123 + A8x91
  ↓
  Argon2id
  ↓
passwordHash
```



Database stores something conceptually like:

```
email: gopi@gmail.com

salt: A8x91

passwordHash: HASH123
```



Later, login

User types:

```
hello123
```



Server retrieves:

```
salt = A8x91  
passwordHash = HASH123
```



Then verifies:

```
hello123 + A8x91  
  ↓  
  Argon2id  
  ↓  
  HASH123
```



Compare:

```
newlyComputedHash === storedHash
```



```
HASH123 === HASH123
```



✅ Correct password.

10. What if they enter the wrong password?

They enter:

```
hello456
```



Server uses the same stored salt:

```
hello456 + A8x91  
  ↓  
  Argon2id  
  ↓  
  HASH999
```



Compare:

```
HASH999 === HASH123
```



❌ Doesn't match.

Login fails.

11. One more important thing: modern password hashes often package the salt

In production, you may not see separate database columns like:

```
salt
passwordHash
```



Many password-hashing libraries return a single encoded string containing parameters, salt, and hash.

Something conceptually like:

```
$argon2id$v=19$m=65536,t=3,p=4$[salt]$[hash]
```



So don't get stuck thinking:

"I always need a `salt` column."

The library may package it into the stored password hash string.

The simplest mental model

Forget all the complicated terminology for a second.

Think of it like this:

SHA-256

A general-purpose hash function:

```
input
↓
SHA-256
↓
hash
```



Very fast → not ideal for password storage.

Salt

A random value added per password:

```
password + random salt
↓
```



Makes identical passwords produce different results.

Argon2id / bcrypt

Password-specific hashing algorithms designed to make guessing expensive:

```
password + salt
  ↓
  Argon2id
  ↓
password hash
```



🎯 And this is what I want you to remember for interviews

Don't say:

"We encrypt the password with SHA-256."

❌ Wrong.

Say:

"We hash passwords using a password-specific hashing algorithm such as Argon2id or bcrypt, with a unique salt for each password."

And if they ask:

"Why not SHA-256?"

Say:

"SHA-256 is a general-purpose hash designed to be fast. Password hashing should deliberately be computationally expensive to make brute-force guessing harder, so we use algorithms such as Argon2id or bcrypt."

That's the core. 🔥

And now the next thing that will make this **really click** is seeing **actual Next.js code using a password-hashing library**, because then you'll see exactly where the salt comes from and whether *you* manually create it or the library does it.





Awesome 😊 Let's move to the **next important Auth.js concept: how the session is actually used to protect pages and APIs.**

Lesson 7.x — Protecting Routes with Auth.js

We've now understood:

```
Google
  ↓
OAuth/OIDC
  ↓
Auth.js callback
  ↓
User/Account
  ↓
Session
  ↓
Cookie
```



Now the natural question is:

Once the user is logged in, how do I actually stop an unauthenticated user from accessing `/dashboard` or `/api/orders` ?

This is where `auth()` becomes really important.

1. Protecting a Server Component

Suppose we have:

```
app/  
├── login/  
│   └── page.tsx  
└── dashboard/  
    └── page.tsx
```

Our dashboard should only be accessible to logged-in users.

We can write:

```
<\/> TypeScript  
  
// app/dashboard/page.tsx  
  
import { auth } from "@auth";  
import { redirect } from "next/navigation";  
  
export default async function DashboardPage() {  
  const session = await auth();  
  
  if (!session) {  
    redirect("/login");  
  }  
  
  return (  
    <div>  
      <h1>Welcome {session.user?.name}<\/h1>  
      <p>{session.user?.email}<\/p>  
    <\/div>  
  );  
}
```

Now let's understand **exactly** what happens.

2. User requests `/dashboard`

Browser:

```
</> http

GET /dashboard
Cookie: authjs.session-token=ABC123
```



Next.js receives this request.

Because `dashboard/page.tsx` is a **Server Component**, it executes on the server.

Then:

```
</> TypeScript

const session = await auth();
```



runs.

3. What does `auth()` do?

Remember where it came from:

```
</> TypeScript

// auth.ts

export const {
  handlers,
  auth,
  signIn,
  signOut
} = NextAuth({...});
```



So `auth()` is an Auth.js server-side function.

Conceptually:

```
Browser
  |
  | Cookie: session-token=ABC123
  ▼
Next.js
  |
  ▼
```



Dashboard Server Component

|
| auth()
▼

Auth.js

|
▼

Session verification / lookup

|
▼

User session

If the session is valid:

⌄ TypeScript



```
session = {  
  user: {  
    name: "Gopi",  
    email: "gopi@example.com"  
  }  
}
```

Then the page renders.

4. What if the user isn't logged in?

Suppose there is no valid session.

Then:

⌄ TypeScript



```
const session = await auth();
```

returns something equivalent to:

⌄ TypeScript



```
null
```

So:

⌄ TypeScript



```
if (!session) {  
  redirect("/login");  
}
```

```
}
```

executes.

Next.js sends a redirect response.

Conceptually:

```
</> http
```



```
307 Temporary Redirect
```

```
Location: /login
```

The browser then requests:

```
</> http
```



```
GET /login
```

So:

```
User → /dashboard
```

```
↓
```

```
auth()
```

```
↓
```

```
no session
```

```
↓
```

```
redirect("/login")
```

```
↓
```

```
/login
```



5. Why isn't this authorization?

This is an important distinction.

This code checks:

```
</> TypeScript
```



```
if (!session)
```

That is **authentication**.

We're asking:

"Is this person logged in?"

But suppose we have:

Admin Dashboard



and only admins can access it.

Then we need **authorization**.

For example:

</> TypeScript



```
const session = await auth();

if (!session) {
  redirect("/login");
}

if (session.user.role !== "ADMIN") {
  redirect("/forbidden");
}
```

Now we're asking two different questions:

Authentication:

"Who are you / are you logged in?"



Authorization:

"Are you allowed to do this?"

6. But there's an important security rule

Never rely only on hiding UI.

For example:

</> TypeScript



```
{session?.user.role === "ADMIN" && (
  <button>Delete User</button>
)}
```

This is useful for UX.

But it is **not security**.

A malicious user can manually send a request to your server.

For example:

```
</> http  
  
POST /api/users/delete
```



Therefore your server-side action/API must check authorization too.

7. Protecting a Route Handler

Suppose we have:

```
app/api/orders/route.ts
```



We can do:

```
</> TypeScript  
  
import { auth } from "@auth";  
import { NextResponse } from "next/server";  
  
export async function POST(request: Request) {  
  const session = await auth();  
  
  if (!session) {  
    return NextResponse.json(  
      { error: "Unauthorized" },  
      { status: 401 }  
    );  
  }  
  
  // User is authenticated  
  // Continue processing order...  
  
  return NextResponse.json({  
    success: true,  
  });  
}
```



Now:

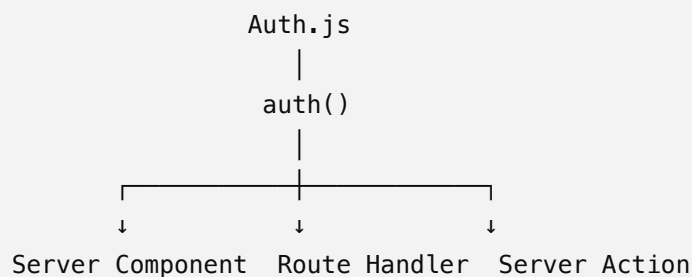
```
Browser  
|
```



```
POST /api/orders
Cookie: session-token=ABC
▼
Next.js Route Handler
|
▼
auth()
├── no session → 401
└── valid session → continue
```

8. This is why `auth()` is so useful

You can use the same server-side authentication context in multiple places:



For example:

Server Component

`</>` TypeScript

```
const session = await auth();
```

Route Handler

`</>` TypeScript

```
const session = await auth();
```

Server Action

`</>` TypeScript

```
const session = await auth();
```

The exact mechanics differ depending on the execution context, but the idea is:

Your server-side code asks Auth.js to resolve the current authenticated user/session.

9. Server Action example

Suppose:

```
</> TypeScript   
"use server";  
  
import { auth } from "@auth";  
  
export async function deleteAccount() {  
  const session = await auth();  
  
  if (!session) {  
    throw new Error("Unauthorized");  
  }  
  
  // Delete account...  
}
```

Even if your UI hides the "Delete Account" button from logged-out users, the Server Action still checks authentication.

That's the security boundary.

10. Interview answer 🎯

If an interviewer asks:

"How would you protect a page in Next.js using Auth.js?"

A strong answer would be:

"For server-rendered pages, I can call Auth.js's server-side `auth()` function to resolve the current session. If there's no valid session, I redirect the user to the login page. For protected mutations or APIs, I perform the authentication and authorization checks again inside the Server Action or Route Handler rather than relying on client-side UI checks."

And if they ask:

"Is hiding an admin button authorization?"

Say:

"No. That's only a UI concern. Authorization must be enforced on the server at the resource or mutation boundary."

One thing I want you to notice

We now have **three different places** where Auth.js can participate:

1. LOGIN
 `signIn()`
 ↓
 `OAuth`
 ↓
 `callback`
 ↓
 `session created`

2. READING SESSION
 `auth()`
 ↓
 `"Who is currently logged in?"`

3. PROTECTING ACTIONS
 `auth()`
 ↓
 `authenticate`
 ↓
 `authorize`
 ↓
 `perform operation`

That distinction is **very interview-important**.

Next after this

The natural next piece is **Auth.js callbacks** — specifically:

⌄ TypeScript

```
callbacks: {  
  async jwt() {},  
  async session() {},  
  async signIn() {},
```

```
async redirect() {}  
}
```

Because you've now seen `session` being created and consumed, the next question is:

"How do I customize what gets stored in the JWT/session — for example, how do I put a user's `role` or `id` into the session?"

That's where the `jwt()` and `session()` callbacks finally make sense. 🔥



how many more are there

Not too many 😊 We're actually pretty close to wrapping the **Authentication & Authorization** section.

Based on what we've covered so far, I'd structure the remaining part like this:

🔒 Remaining Auth.js / Auth & Authorization lessons

#	Topic	What you'll learn
7.6	🛡️ Route Protection	<code>auth()</code> , protecting pages, APIs, Server Actions
7.7	🔄 Auth.js Callbacks	<code>signIn</code> , <code>jwt</code> , <code>session</code> , <code>redirect</code> callbacks
7.8	👤 Session Customization	Put <code>user.id</code> , <code>role</code> , etc. into the session
7.9	🔑 Authorization / RBAC	Roles, permissions, admin/user access
7.10	🍪 Session Strategies	Database sessions vs JWT sessions
7.11	🔒 CSRF + SameSite	Why state-changing requests need protection
7.12	🌐 Middleware / Proxy Auth	Protecting groups of routes before rendering
7.13	📄 Logout & Session Expiry	Sign-out, expiration, invalid/expired sessions

#	Topic	What you'll learn
7.14	 Complete Auth Architecture	Put Browser → Next.js → Auth.js → OAuth → DB → Session together
7.15	 Auth Interview Questions	Real interview scenarios + architecture questions

So **about 10 lessons remain**, depending on how deeply we split things.

But some of these are short. The **big ones** are:

- 1. **Callbacks**
- 2. **Session customization**
- 3. **Authorization/RBAC**
- 4. **JWT vs DB sessions**
- 5. **CSRF/SameSite**
- 6. **Middleware/Proxy**

After that, I'd consider the Auth section **complete** and move you to the next major Next.js/interview area. 🚀

And I wouldn't rush through these—especially callbacks and authorization. Given the way you've been learning, I'd rather make sure you can explain the **actual request flow** than just memorize Auth.js APIs.



so after this authentication and authorization are done?

Yes. 🙌 **After we finish those remaining Auth.js topics, we can consider the Authentication & Authorization section complete.**

More precisely, your **Next.js Auth & Authorization roadmap** will cover:

AUTHENTICATION



- ✓ Authentication vs Authorization
- ✓ Auth architecture
- ✓ Cookies
- ✓ Sessions
- ✓ JWT

- ✓ Password hashing
- ✓ Login flow
- ✓ Server Components + auth
- ✓ Client Components + auth
- ✓ Auth.js
- ✓ OAuth 2.0
- ✓ OpenID Connect
- ✓ Google authentication
- ✓ Authorization code + callback
- ✓ Client ID / Client Secret
- ✓ Token endpoint
- ✓ Auth.js adapters
- ✓ ORM + database persistence
- ✓ User / Account / Session
- ✓ Database sessions vs JWT sessions
- ✓ Route protection
- ✓ Auth.js callbacks
- ✓ Session customization

AUTHORIZATION

- ✓ Authentication vs authorization
- ✓ Protecting Server Components
- ✓ Protecting Route Handlers
- ✓ Protecting Server Actions
- ✓ Roles / RBAC
- ✓ Permissions
- ✓ Server-side authorization
- ✓ Client UI vs actual security boundary

SECURITY

- ✓ HttpOnly / Secure / SameSite
- ✓ CSRF
- ✓ Session expiry
- ✓ Logout
- ✓ Middleware / Proxy protection
- ✓ Common auth vulnerabilities

Then we'll do **one complete end-to-end architecture walkthrough**:

Browser

↓

Login

↓

Auth.js

↓

Google OAuth/OIDC

↓



```
Callback
  ↓
Google token exchange
  ↓
User + Account
  ↓
Adapter
  ↓
Database
  ↓
Session
  ↓
Cookie
  ↓
Protected request
  ↓
auth()
  ↓
Authentication
  ↓
Authorization
  ↓
Protected resource
```

That final walkthrough is important because you've been asking the right kind of **"but who actually calls what?"** questions. 😊 It will tie all the individual pieces together.

So yes: once we finish these, you can confidently mark *Next.js Authentication & Authorization* as DONE. 🗝️✅



ChatGPT can make mistakes. Check important info.